Université de Tunis El Manar
Faculté des Sciences de Tunis
Master de recherche en Physique quantique

Compte rendu
Physique numérique
2026–2027

Période du pendule simple non linéaire

Intégration numérique par Runge–Kutta d'ordre 4 en Python

Simulation n° 1 | Étudiant Ahmed Rekik | Enseignant Nom de l'enseignant | Date 31 octobre 2026

1 Objectif

Calculer numériquement la période T d'un pendule simple lâché sans vitesse d'un angle θ_0 quelconque, comparer le résultat à la valeur exacte et au développement en série, puis vérifier l'ordre de convergence de la méthode de Runge–Kutta d'ordre 4 (RK4) utilisée.

2 Modèle physique

L'angle θ d'un pendule simple de pulsation ω_0 obéit à $\ddot{\theta} + \omega_0^2 \sin\theta = 0$. Avec le temps réduit $\tau = \omega_0 t$, l'équation devient sans dimension :

$$\frac{\mathrm{d}^2\theta}{\mathrm{d}\tau^2} = -\sin\theta, \qquad \theta(0) = \theta_0, \quad \frac{\mathrm{d}\theta}{\mathrm{d}\tau}(0) = 0. \tag{2.1}$$

L'énergie réduite $\varepsilon = \frac{1}{2}\left(\frac{\mathrm{d}\theta}{\mathrm{d}\tau}\right)^2 - \cos\theta$ est conservée, ce qui fournit un test de la simulation. La période exacte s'exprime à l'aide de l'intégrale elliptique complète K ou de la moyenne arithmético-géométrique AGM [1] :

$$\frac{T}{T_0} = \frac{2}{\pi} K\!\left(\sin\tfrac{\theta_0}{2}\right) = \frac{1}{\mathrm{AGM}\left(1, \cos\frac{\theta_0}{2}\right)}, \qquad T_0 = 2\pi. \tag{2.2}$$

Pour les petites amplitudes, le développement en série s'écrit

$$\frac{T}{T_0} = 1 + \frac{\theta_0^2}{16} + \frac{11\,\theta_0^4}{3072} + \mathcal{O}\!\left(\theta_0^6\right). \tag{2.3}$$

3 Méthode numérique

On pose $\mathbf{y} = (\theta, \mathrm{d}\theta/\mathrm{d}\tau)$ et $\mathbf{f}(\mathbf{y}) = (y_2, -\sin y_1)$. Un pas de RK4 de longueur h est

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{h}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right), \tag{3.1}$$

$$\begin{aligned} \mathbf{k}_1 &= \mathbf{f}(\mathbf{y}_n), & \mathbf{k}_2 &= \mathbf{f}\left(\mathbf{y}_n + \tfrac{h}{2}\mathbf{k}_1\right), \\ \mathbf{k}_3 &= \mathbf{f}\left(\mathbf{y}_n + \tfrac{h}{2}\mathbf{k}_2\right), & \mathbf{k}_4 &= \mathbf{f}\left(\mathbf{y}_n + h\,\mathbf{k}_3\right). \end{aligned} \tag{3.2}$$

L'erreur globale de la méthode est en $\mathcal{O}(h^4)$. La période vaut quatre fois la durée du premier passage par $\theta = 0$, localisée par bissection sur le dernier pas (programme 1).

4 Implémentation

Le programme 1 est extrait directement du fichier `code/simulation.py` : toute modification du script est donc reprise à la prochaine compilation. Le script complet (bibliothèque standard de Python uniquement) écrit les fichiers `data/*.csv` lus ci-dessous.

Programme 1. Pas RK4 et calcul de la période (`code/simulation.py`).

```
def rk4(theta, omega, h):
    """Un pas de Runge-Kutta d'ordre 4 pour (theta, omega)."""
    k1t, k1w = omega, accel(theta)
    k2t, k2w = omega + 0.5 * h * k1w, accel(theta + 0.5 * h * k1t)
    k3t, k3w = omega + 0.5 * h * k2w, accel(theta + 0.5 * h * k2t)
    k4t, k4w = omega + h * k3w, accel(theta + h * k3t)
    return (theta + h / 6 * (k1t + 2 * k2t + 2 * k3t + k4t),
            omega + h / 6 * (k1w + 2 * k2w + 2 * k3w + k4w))


def periode(theta0, h):
    """Période = 4 fois la durée du premier passage par theta = 0 (lâché sans \
vitesse)."""
    t, theta, omega = 0.0, theta0, 0.0
    while True:
        suivant = rk4(theta, omega, h)
        if suivant[0] <= 0.0:                    # zéro dans [t, t + h]
            bas, haut = 0.0, h                   # bissection sur le pas partiel
            for _ in range(60):
                milieu = 0.5 * (bas + haut)
                if rk4(theta, omega, milieu)[0] > 0.0:
                    bas = milieu
                else:
                    haut = milieu
            return 4.0 * (t + 0.5 * (bas + haut))
        t, (theta, omega) = t + h, suivant
```

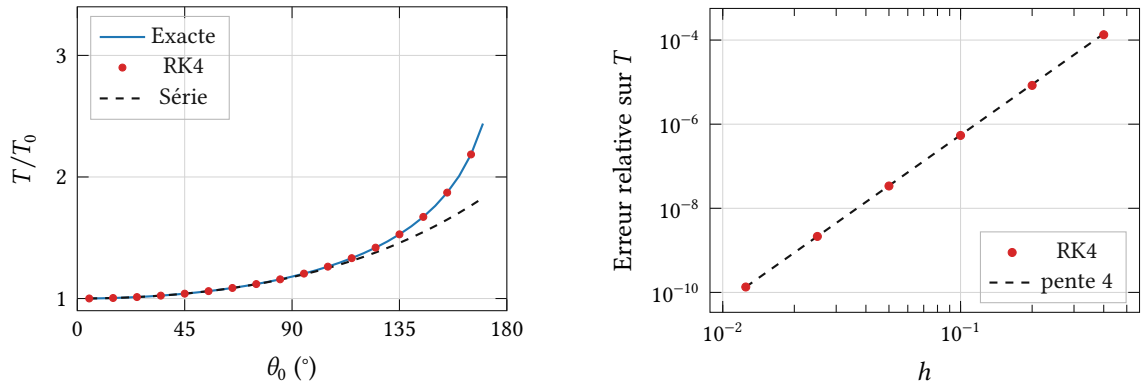
5 Résultats expérimentaux

5.1 Période en fonction de l'amplitude

Le tableau 1 compare la période calculée avec un pas $h = 0{,}01$ à la valeur exacte (2.2) et au développement (2.3). Pour $\theta_0 = 90°$, on obtient $T/T_0 = 1{,}18034$.

Tableau 1. Période réduite T/T_0 (données : `data/tableau_periode.csv`).

θ_0 (°)	RK4	Exacte	Série (2.3)	Erreur relative RK4
10	1,001907	1,001907	1,001907	$8{,}2 \times 10^{-11}$
30	1,017409	1,017409	1,017404	$7{,}4 \times 10^{-11}$
60	1,073182	1,073182	1,072845	$5{,}5 \times 10^{-11}$
90	1,180341	1,180341	1,176012	$3{,}9 \times 10^{-11}$
120	1,372881	1,372881	1,343054	$3{,}7 \times 10^{-11}$
150	1,762204	1,762204	1,596576	$4{,}9 \times 10^{-11}$
170	2,439363	2,439363	1,827724	$6{,}1 \times 10^{-11}$



(a) Période réduite.

(b) Convergence en h ($\theta_0 = 60°$).

Figure 1. Résultats de la simulation, tracés directement depuis les fichiers CSV : (a) période en fonction de l'amplitude ; (b) erreur relative de RK4 en fonction du pas, en échelle logarithmique.

6 Validation et convergence

Pour $\theta_0 = 60°$, l'erreur relative décroît de $1{,}3 \cdot 10^{-4}$ ($h = 0{,}4$) à $1{,}3 \cdot 10^{-10}$ ($h = 0{,}0125$). La pente de $\log(\text{erreur})$ en fonction de $\log h$, ajustée sur les six pas, vaut

$$p = 3{,}98, \tag{6.1}$$

en accord avec l'ordre théorique 4 de RK4 (figure 1b).

7 Analyse et discussion

Qualité de la simulation. Avec $h = 0{,}01$, l'erreur relative sur T reste inférieure à $8 \cdot 10^{-11}$ pour toutes les amplitudes de 5° à 170° : la précision est limitée par l'arithmétique en double précision plutôt que par la méthode.

Validité du développement en série. La série (2.3) reproduit la période à mieux que $1 \cdot 10^{-3}$ jusqu'à environ 60°, mais s'en écarte fortement ensuite : pour $\theta_0 = 170°$ elle donne $T/T_0 = 1{,}83$ contre 2,44 pour la valeur exacte, soit une sous-estimation d'environ 25 %. La période diverge lorsque $\theta_0 \to \pi$ (pendule lâché au voisinage de la position d'équilibre instable), ce qu'aucun développement polynomial ne peut représenter.

8 Conclusion

L'intégration RK4 reproduit la période exacte du pendule non linéaire avec un ordre de convergence mesuré $p = 3{,}98$. Le développement en série n'est fiable que pour les amplitudes modérées. La chaîne « script Python → fichiers CSV → document LaTeX » garantit que les tableaux, les courbes et les valeurs citées proviennent toujours de la dernière exécution du programme.

Références

[1] L. D. Landau et E. M. Lifshitz, *Mechanics*, 3e éd., t. 1, Course of Theoretical Physics (Butterworth-Heinemann, 1976).